IST 387: Introduction to Applied Data Science
Lab 2: Part 1 Notes

Notes Prepared by: Dr. Preeti Jagadev

Reference: Data Science for Business with R, written by Jeffrey S. Saltz and Jeffrey M. Stanton

Contents of the Notes:

- Introduction to variables, cases, and case labels in a Dataframe.
- Creating Dataframes
- Exploring Dataframes
- Accessing Columns in a Dataframe
- Case Study: Calculating NPS using a Dataframe
- R Functions used in this chapter

Preeti Jagadev

Introduction

❖ Variable/Attribute:

Consider the **table 1** shown below. This table is a two-dimensional representation of rows and columns. Here, the top line in bold is not really part of the data. Instead, the top line contains the **attribute** or **variable** names. For example, **Age** is an **attribute** that every living thing has, and you could count it in minutes, hours, days, months, years, or other units of time. Here we have the **Age** attribute calibrated in years. Imagine how much more difficult it would be to understand what was going on in this table, without the attribute or variables mentioned at the top.

Table 1: My family data frame

Name	Age	Gender	Weight
Dad	43	Male	188
Mom	42	Female	136
Sis	12	Female	83
Bro	8	Male	61
Dog	5	Female	44

❖ Instance/Case:

Other than the top row (which contains the variable names), each of the remaining row is an **instance** or a **case.**

❖ Case label:

You could think of the **Name column** as comprising of **case labels**, where each of these labels refers to one and only one row in our data. The case label is unique for each case (i.e., the same case label would never appear in more than one row). For e.g., the **Dad** row can be distinguished from the **Bro** row, even though both are males. Even if we add an **Uncle** row that has the same **Age, Gender**, and **Weight** as **Dad**, we will still be able to tell the two rows apart because one would have the name **Dad** and the other would have the name **Uncle**.

Rows and Columns can be organized any way that you want, but the most common way is to have the **rows** be **cases or instances**, and the **columns** be **attributes or variables**.

One other important note: Look at how each column contains the same kind of data all the way down. For example, the **Age** column is all numbers. There is nothing in that **Age** column like **Old** or **Young.** This is a valuable way of keeping things organized. After all we cannot run the **mean()** function on the **Age** column if it contains a little piece of text, like **Old** or **Young**.

Finally, see that every column has the same number of entries, so that he whole forms a nice rectangle. When statisticians and other people who work with databases work with a dataset, they expect this rectangular arrangement.

Creating Dataframes

Let's figure out how to get these rows and columns into R. Let's start by building each column one by one and then join them together. This is somewhat labor intensive, and not the usual way we would work with a dataset, but it is easy to understand.

Preeti Jagadev

First, we run this command to make the column of names:

```
myFamilyNames <- c("Dad", "Mom", "Sis", "Bro", "Dog")
```

Please note, every name is placed within double quotes. This is a way of signaling R that you want it to treat something as a **string** of characters, and not as the name of a storage location. If we had asked R to use Dad instead of "Dad", then it would have looked for a storage location (a data object or variable) named Dad.

Another thing to notice is that the commas separating the different values are outside of the double quotes. This is because the comma can only do its job of separating the different values if it is not included inside the quotes.

Let's check the contents of myFamilyNames, as follows:

```
myFamilyNames
```

The output is as follows:

```
[1] "Dad" "Mom" "Sis" "Bro" "Dog"
```

Next, create a vector of the Age of the family members, as follows:

```
myFamilyAges <- c(43, 42, 12, 8, 5)
```

The output is as follows:

```
[1] 43 42 12  8  5
```

Next, create a vector of the Gender of the family members, as follows:

```
myFamilyGenders <- c("Male", "Female", "Female", "Male", "Female")
```

The output is as follows:

```
[1] "Male"   "Female" "Female" "Male"   "Female"
```

Next, create a vector of the Weight of the family members, as follows:

```
myFamilyWeights <- c(188, 136, 83, 61, 44)
```

The output is as follows:

```
[1] 188 136  83  61  44
```

As observed above, our dataframe consists of four variables, each with five values in it. Two of the variables are character data, and the other two are integer data.

Preeti Jagadev

Now, let's explore the variable myFamilyAges a little further. We now know that myFamilyAges is a variable and is a list of numbers. We can access each number individually, using square brackets. For example, if we want to output just the second element in myFamilyAges, we could do the following:

```
myFamilyAges[2]
```

The output will be:

```
[1] 42
```

Now, we are ready to tackle the dataframe. In R, a dataframe is a list of elements (columns), where each element (column) in the list is a vector. Each vector is the same length, which is how we get our nice rectangular row and column setup, and generally each vector has its own name.

The command to make the dataframe is very simple as follows:

```
myFamily <- data.frame(myFamilyNames, myFamilyAges, myFamilyGenders, myFamilyWeights)
```

The **data.frame()** function makes a dataframe from the four vectors that we listed. Notice that we have also used the assignment arrow to make a new stored location where R puts the dataframe. This new data object called myFamily, is our dataframe.
If we type myFamily in the command line, we get the output as follows:

```
  myFamilyNames myFamilyAges myFamilyGenders myFamilyWeights
1           Dad           43            Male             188
2           Mom           42          Female             136
3           Sis           12          Female              83
4           Bro            8            Male              61
5           Dog            5          Female              44
```

Notice that R has put row numbers in front of each row of our data, i.e., actual indices of the dataframe. Later, we will use row numbers like these to extract single rows or groups of rows from dataframes.

Exploring Dataframes

With a small dataset like this one, only five rows, it is easy just to look at all the data. But when we get to a bigger dataset, this won't be practical. We need to have other ways of summarizing what we have.

The method shown below reveals the type of structure that R has used to store a data object.

```
str(myFamily)
```

The output is as follows:

```
'data.frame':   5 obs. of  4 variables:
 $ myFamilyNames  : chr  "Dad" "Mom" "Sis" "Bro" ...
 $ myFamilyAges   : num  43 42 12 8 5
 $ myFamilyGenders: chr  "Male" "Female" "Female" "Male" ...
 $ myFamilyWeights: num  188 136 83 61 44
```

Here, the function **str()** reveals the structure of the data object that you name between the parentheses. In this case we know that **myFamily** is a dataframe because we just set that up in the previous command. In the future, however, we will run into many situations where we are not sure how R has created a data object, so it is important to know **str()** so that you can ask R to report what an object is at any time.

In the first line of the output, we have the confirmation that **myFamily** is a dataframe as well as an indication that there are 5 observations (obs.) and four variables.

After the first line of output, we have four sections that each begin with $. For each of the four variables, these sections describe the component columns of the **myFamily** dataframe object.

Each of the four variables has a mode or type that is reported by R, right after the colon on the line that names the variable.

Next, we will examine another very useful function called **summary ()**.

```
summary(myFamily)
```

The summary command provides some overlapping information to the str command but also goes a little further, particularly with numeric variables. Here is what we get at the output:

```
 myFamilyNames       myFamilyAges  myFamilyGenders    myFamilyWeights
 Length:5           Min.   : 5    Length:5           Min.   : 44.0
 Class :character   1st Qu.: 8    Class :character   1st Qu.: 61.0
 Mode  :character   Median :12    Mode  :character   Median : 83.0
                    Mean   :22                       Mean   :102.4
                    3rd Qu.:42                       3rd Qu.:136.0
                    Max.   :43                       Max.   :188.0
```

It can be observed that the name of a column/variable sits up above the information that pertains to it. Notice, as with str (), that the output is quite different depending on whether we are talking about a variable, like **myFamilyNames** or **myFamilyGenders**, versus a numeric variable like **myFamilyAges** and **myFamilyWeights.**

For the numeric variables we get five different calculated quantities that help to summarize the variable, as follows:

- **Min:** Refers to the minimum or lowest value among all the cases. For the present dataframe, five is the age of the dog and it is the lowest age of all the family members.
- **Median:** Median refers to the value of the case that splits the whole group in half, with half of the cases having higher values, and half having lower values.

Example 1:

- Consider the data set {3, 13, 2, 34, 11, 26, 47}.
- Arrange these data points in ascending order, i.e. {2, 3, 11, 13, 26, 34, 47}.
- The number of data points is 7, which is an **odd number.** Thus, the median is the number in the middle of {2, 3, 11, 13, 26, 34, 47}. which in this instance is 13 since there are three numbers on either side.

Example 2:

- Consider the data set {3,7,9,5}.
- Arrange these data points in ascending order, i.e. {3,5,7,9}.
- The number of datapoints is 4, which is an **even number**. If there is an even number of observations, then there is no single middle value. Thus, the median is defined to be the mean of the two middle values: so the median of 3, 5, 7, 9 is (5+7)/2 = 6.

- **1st Qu:** 1st Qu refers to **first quartile (Q1).** If we take all the cases and arrange them in ascending order, we can then divide up the whole into four groups, where each group has the same number of observations. Consider the examples given below to understand the concept of 1st Qu.

Example 1:

- Consider the data set {1, 3, 5, 5, 5, 6, 7, 7, 7, 9, 10}.
- These data points are already in ascending order.
- The number of datapoints is 11, which is an **odd number**. Thus, the median is 6.
- Q1 is the data point exactly halfway in the lower half of the data set. Find it by finding the median of the data below the median. Thus, in this case, Q1 is 5.

  1,3,5,5,5,6,7,7,7,9,10

Example 2:

- Consider the data set {5, 7, 4, 4, 6, 2, 8}.
- Arrange these data points in ascending order, i.e. {2, 4, 4, 5, 6, 7, 8}.
- The number of datapoints is 7, which is an odd number. Thus, the median is 5.
- Q1 is the data point exactly halfway in the lower half of the data set. Find it by finding the median of the data below the median. Thus, in this case, Q1 is 4.

  2, 4, 4, 5, 6, 7, 8

- **3rd Qu:** 3rd Qu refers to **third quartile (Q3).** Consider the examples given below to understand the concept of 3rd Qu.

Example 1:

- Consider the data set {1, 3, 5, 5, 5, 6, 7, 7, 7, 9, 10}.
- These data points are already in ascending order.
- The number of datapoints is 11, which is an **odd number**. Thus, the median is 6.
- Q3 is the data point exactly halfway in the upper half of the data set. Find it by finding the median of the data above the median. Thus, in this case, Q3 is 7.

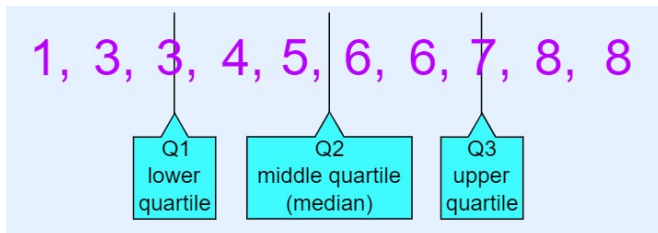
  1,3,5,5,5,6,7,7,7,9,10

Preeti Jagadev

Example 2:

- Consider the data set { 1, 3, 3, 4, 5, 6, 6, 7, 8, 8}.
- These data points are already in ascending order.
- The number of datapoints is 10, which is an **even number**. Thus, the median is (5+6)/2 = 5.5.
- Q3 is the data point exactly halfway in the upper half of the data set. Find it by finding the median of the data above the median. Thus, in this case, Q3 is 7.

  1, 3, 3, 4, 5, 6, 6, 7, 8, 8



- **Mean:** It is the numeric average of all the values. For instance, the average age in the family is 22.
- **Max:** It is the maximum value among all the available cases. For example, in this dataframe, Dad has the highest weight: 188.

While both the str and summary functions are very useful, sometimes we just want to look at a couple of rows in the dataframe. If the dataframe has many rows, a better way is to use head or tail:

Given below is the head command:

```
head(myFamily, 2)
```

The output is as follows:

```
  myFamilyNames myFamilyAges myFamilyGenders myFamilyWeights
1           Dad           43            Male             188
2           Mom           42          Female             136
```

Given below is the tail command:

```
tail(myFamily, 2)
```

The output is as follows:

```
  myFamilyNames myFamilyAges myFamilyGenders myFamilyWeights
4           Bro            8            Male              61
5           Dog            5          Female              44
```

You can see in the code that head () lists the first rows in the dataframe and tail () lists the last rows in the dataframe. The number of rows to be displayed is the second argument to both head ()

and tail (). In our case we used head () to show the first two rows and tail () to show the last two rows in the myFamily dataframe.

Accessing Columns in a Dataframe

R stores the dataframe as a list of vectors. We can use the name of the dataframe together with the name of a vector and use the $ to connect the two labels like this.

```
myFamily$myFamilyAges
```

The output will be as follows:

```
[1] 43 42 12  8  5
```

Now, you may wonder why we did not directly refer to myFamilyAges, instead of typing this command. Well, this is a very important point. When we created the myFamily dataframe, we copied all the information from the individual vectors, into this storage space.

You can prove this to yourself very easily, by adding some data to the original vector, myFamilyAges:

```
myFamilyAges <- c(myFamilyAges, 11)
myFamilyAges
```

We use the c() command to add the value 11 to the original list of ages that we had stored in myFamilyAges The output will be as follows:

```
[1] 43 42 12  8  5 11
```

Now, lets type the command:

```
myFamily$myFamilyAges
```

The output remains unchanged, and is as follows:

```
[1] 43 42 12  8  5
```

This shows that the dataframe and its component vectors is now completely independent piece of data.

Thus, we must be very careful, if we establish a dataframe that we want to use for subsequent analysis, that we don't make a mistake and keep using some of the original data from which we assembled the dataframe.

Here's a puzzle that follows on from this question. We have a nice dataframe with five observations and four variables. This is a rectangular dataset. What we tried to add a new piece of data on the end of one of the variable? In other words, what if we tried something like the following command?

```
myFamily$myFamilyAges <- c(myFamily$myFamilyAges, 11)
```

The output is as follows:

Preeti Jagadev

```
Error in `$<-.data.frame`(`*tmp*`, myFamilyAges, value = c(43, 42, 12,  :
  replacement has 6 rows, data has 5
```

The error occurs because the variables in the dataframe that contain the family member's ages, all of a sudden have one more observation than the other variables: no more perfect rectangle.

Case Study: Calculating NPS using a Dataframe

Let's practice working with dataframes by considering a small number of survey responses.

There are six surveys, where the *likelihood to recommend (LTR)* values are 9,9,7,6,8,7 by people traveling in "Business travel", "Business travel", "Mileage tickets", "Personal travel", and "Personal travel", respectively. Given this, is there a difference in *net promoter score (NPS),* comparing all the survey responses to just the business travel tickets?

Here is the code for all the survey responses:

```r
# Six surveys
ltr <- c(9,9,7,6,8,7)

typeOfTravel <- c("Business travel",
                  "Business travel","Business travel","Mileage",
                  "Personal travel","Personal travel")

survey <- data.frame(ltr, typeOfTravel)

#Calculate number of promoters and detractors
numP <- sum(survey$ltr > 8)

numD <- sum(survey$ltr < 7)

#calculate NPS
total <- nrow(survey)

nps <- (numP/total - numD/total) * 100

nps
```

The output is obtained as follows:

```
> nps
[1] 16.66667
```

Now, do the same analysis for the business travel tickets:

```
busTravelDF <- survey[survey$typeOfTravel=="Business travel",]

#Calculate number of promoters and detractors
numP <- sum(busTravelDF$ltr > 8)

numD <- sum(busTravelDF$ltr < 7)

#calculate NPS
total <- nrow(busTravelDF)

bus.nps <- (numP/total - numD/total) * 100

bus.nps
```

The output is obtained as follows:

```
> bus.nps
[1] 66.66667
```

If we want to check the structure of survey, then we do it as follows:

```
str(survey)
```

The output is obtained as follows:

```
> str(survey)
'data.frame':   6 obs. of  2 variables:
 $ ltr         : num  9 9 7 6 8 7
 $ typeOfTravel: chr  "Business travel" "Business travel" "Business travel" "Mileage" ...
```

We can also check the structure of the following:

```
str(survey$typeOfTravel)
```

The output is as follows:

```
 chr [1:6] "Business travel" "Business travel" "Business travel" "Mileage" "Personal travel" ...
```

R Functions used in this chapter

- c ( ): Combines data elements together.
- <- : Indicates an assignment arrow.
- data.frame ( ): Makes a dataframe from separate vectors.
- head ( ) : List the first rows in a dataframe.
- str ( ): Reports the structure of a data object.
- summary ( ): Reports data types and a data overview.
- tail ( ): Lits the last rows in a dataframe.
- nrow ( ): Reports the number of rows for a dataframe.
- sum( ): Calculates the sum of a set of values.